TD 4 – Complexité Algorithmique

Licence 2 Informatique

Année académique 2026/2027

Exercice 1. Démontrer que :

- $n^2 \in O(10^{-5}\, n^3)$
- $25n^4 - 19n^3 + 13n^2 \in O(n^4)$
- $2^{n+10} \in O(2^n)$
- Pour toutes fonctions f et g de $\mathbb{N}$ dans $\mathbb{R}^+$, $f(n) + g(n) \in \Theta\big(\max(f(n), g(n))\big)$
- Pour toutes fonctions f et g de $\mathbb{N}$ dans $\mathbb{R}^+$, $O\big(f(n) + g(n)\big) = O\big(\max(f(n), g(n))\big)$

Exercice 2. Soit T un tableau de n entiers et x un autre entier quelconque.

1/ Écrire un algorithme de complexité $\Theta(n^2)$ permettant de déterminer s'il existe (ou non) deux indices distincts i et j tels que $T[i] + T[j] = x$.

2/ Peut-on aussi affirmer que cet algorithme est en $O(n^2)$?

3/ On suppose que l'on dispose d'une méthode de tri en $O(n \log(n))$ permettant de trier T en ordre croissant. En déduire un algorithme de complexité globale aussi $O(n \log(n))$ répondant à la question 1.

Exercice 3. Soit T un tableau de n entiers contenant un tas, et A sa représentation sous forme d'arbre binaire. On note $h(A)$ la hauteur de A.

1/ Montrer que le plus petit nombre de nœuds que peut contenir A est égal à : $2^{h(A)}$.

2/ Montrer que le plus grand nombre de nœuds que peut contenir A est égal à : $2^{h(A)+1} - 1$.

3/ En déduire que $h(A) = \lfloor \log_2(n) \rfloor$.

4/ On rappelle l'algorithme de tri par tas ci-dessous.

Algorithme 1 : TriTas(T, n)

```
k ← n ;
tant que k > 1 faire
    min ← Suppression(T, k) ;
    k ← k − 1 ;
    T[k + 1] ← min ;
```

(pour la fonction Suppression(T, k) se référer au cours).
Montrer qu'à chaque itération k, le nombre de comparaisons est au pire en $\Theta(\log_2(k))$.

5/ En déduire la complexité du tri par tas au pire des cas.

Exercice 4. On désigne par $C(n)$ la complexité du tri fusion où n est la taille du tableau à trier. n est supposé être une puissance de 2. $C(n)$ désigne le nombre de comparaisons faites dans la procédure de fusion entre les deux sous-tableaux que l'on fusionne. Compte tenu de la récursivité dans l'algorithme, on sait que :

$$C(n) = 2\,C(n/2) + n.$$

Montrer que $C(n) \in \Theta(n \log(n))$.

Exercice 5. On considère un tableau trié en ordre croissant, de n éléments, dans lequel on cherche un élément X quelconque. On utilise pour cela la recherche dichotomique suivante :

Algorithme 2 : Recherche dichotomique

```
Données : T : tableau trié en ordre croissant
Résultat : Position de X ou 0 si X n'est pas dans T
a ← 1,   b ← n,   trouve ← faux ;
tant que (trouve = faux) et (b − a) ≥ 0 faire
    m ← ⌊(a+b)/2⌋ ;
    si T[m] = X alors
        trouve ← vrai ;
    sinon si T[m] > X alors
        b ← m − 1 ;
    sinon
        a ← m + 1 ;
```

1/ Combien d'itérations, au maximum, l'algorithme effectuera si $n = 2^k$ où k est un entier positif ?

2/ On suppose maintenant que T n'est pas forcément trié. Écrire un algorithme de complexité $O(n)$ permettant de trouver X.

Exercice 6 [Tri par insertion récursif]. On considère la procédure récursive suivante permettant de trier un tableau L, de n entiers, en ordre croissant.

Algorithme 3 : TriInsertion(L, n)

```
si n > 1 alors
    TriInsertion(L, n − 1) ;
    Insertion(L, n) ;
```

« Insertion(L, n) » est un appel à une autre procédure de prototype général : Insertion(T, n). Cette dernière prend en argument un tableau T de n éléments. Les valeurs de $T[1]$ à $T[n-1]$ sont supposées déjà triées. Elle insère $T[n]$ à sa bonne place, de sorte que T soit entièrement trié (des indices 1 à n).

1/ Rappeler le pseudo-code de la procédure Insertion(T, n).

2/ Exprimer en fonction de n le nombre de comparaisons effectuées, au pire des cas, par celle-ci pour réaliser l'insertion. On le notera $I(n)$.

3/ Soit $C(n)$ le nombre total de comparaisons faites dans TriInsertion(L, n). Écrire une relation de récurrence dans laquelle $C(n)$ est exprimé en fonction de $C(n-1)$ et $I(n)$.

4/ En déduire que si $I(n) = n - 1$, pour tout $n \geq 1$, alors $C(n) = \dfrac{n(n-1)}{2}$.

5/ Montrer ensuite que $C(n) \in \Theta(n^2)$.

6/ En déduire que $C(n) \in O(n^2)$.